

Proyecto 2: Sonda de Kernel en C y Daemon en Go para la Telemetría de Contenedores
Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Ingeniería en Ciencias y Sistemas
Sistemas Operativos 1
Carnet: 201700698
Nombre: Tulio Pirir
Fecha: Septiembre 2026

GUÍA DE INSTALACIÓN Y EJECUCIÓN

Proyecto 2 - Sistema de Telemetría de Contenedores

REQUISITOS PREVIOS

1.1 Sistema operativo

- Ubuntu 20.04 LTS o superior
- O WSL2 con kernel custom compilado (~/.wsl-kernel)

1.2 Software requerido

Software	Versión mínima	Uso
gcc + make	—	Compilar el módulo kernel
linux-headers	Del kernel en ejecución	Compilar módulos
Go	1.20+	Compilar el daemon
Docker	20.10+	Contenedores
Docker Compose	v2	No estrictamente necesario
Valkey o Redis	7.0+	Base de datos
Grafana	10+	Dashboard

jq	—	Parsear JSON
redis-tools	—	redis-cli

INSTALACIÓN DE DEPENDENCIAS

2.1 Actualizar el sistema

```
bash
sudo apt update
sudo apt upgrade -y
```

2.2 Instalar herramientas base

```
bash
sudo apt install -y \
    build-essential \
    linux-headers-$(uname -r) \
    git \
    curl \
    jq \
    redis-tools \
    bc \
    cron
```

Nota WSL2: si usas kernel custom, linux-headers-\$(uname -r) puede no existir. En ese caso ajustar el KDIR del Makefile al path del kernel custom (/home/tj/wsl-kernel).

2.3 Instalar Go

```
bash
# Verificar si ya está instalado
go version
```

```
# Si no lo está
sudo apt install -y golang-go
```

2.4 Instalar Docker

```
bash
sudo apt install -y docker.io
```

```
# Arrancar
sudo service docker start
```

```
# Agregar usuario al grupo docker (evita sudo)
sudo usermod -aG docker $USER
```

```
# Aplicar el cambio (o cerrar y reabrir terminal)
newgrp docker
```

```
# Verificar
docker ps
docker run --rm hello-world
```

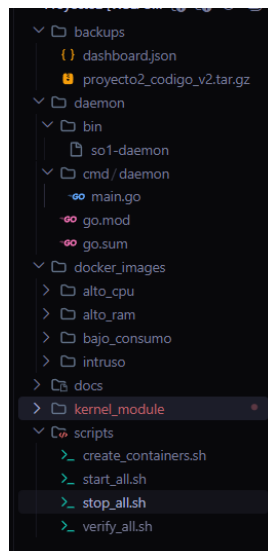
2.5 Verificar cron

```
bash
sudo service cron start
pgrep -x cron
Debe salir un PID.
```

ESTRUCTURA DEL PROYECTO

```
bash
mkdir -p ~/workspaces/SistemasOperativos1/Proyecto2
cd ~/workspaces/SistemasOperativos1/Proyecto2
```

```
mkdir -p kernel_module
mkdir -p daemon/cmd/daemon daemon/bin
mkdir -p docker_images/{bajo_consumo,alto_ram,alto_cpu,intruso}
mkdir -p scripts
mkdir -p docs
mkdir -p backups
Estructura esperada:
```



MÓDULO DE KERNEL

4.1 Crear kernel_module/continfo.c

bash

cd ~/workspaces/SistemasOperativos1/Proyecto2/kernel_module

nano continfo.c

Pega el código del módulo (versión con timestamp). Guardar con Ctrl+O, Enter, Ctrl+X.

4.2 Crear kernel_module/Makefile

bash

nano Makefile

Pega:

makefile

obj-m += continfo.o

KDIR := /home/tj/wsl-kernel

PWD := \$(shell pwd)

PROC_PATH := /proc/continfo_pr2_so1_201700698

all:

\$(MAKE) -C \$(KDIR) M=\$(PWD) KBUILD_MODPOST_WARN=1 modules

clean:

\$(MAKE) -C \$(KDIR) M=\$(PWD) clean

@rm -f *.o *.ko *.mod *.mod.c *.symvers *.order *.cmd

load: all

sudo insmod continfo.ko

@echo "Modulo cargado"

unload:

-sudo rmmod continfo 2>/dev/null || true

@echo "Modulo removido"

Importante: las líneas de los comandos van con TAB, no espacios. Verifica con:

bash

cat -A Makefile | head -10

Debes ver ^I al inicio de los comandos.

4.3 Compilar

```
bash
make clean
make
Debe aparecer: continfo.ko en la carpeta.
bash
ls -la continfo.ko
```

4.4 Cargar

```
bash
sudo insmod continfo.ko
lsmod | grep continfo
ls -la /proc/continfo_pr2_so1_201700698
```

4.5 Verificar

```
bash
cat /proc/continfo_pr2_so1_201700698 | head -30
cat /proc/continfo_pr2_so1_201700698 | jq '.processes | length'
```

4.6 Descargar

```
bash
sudo rmmod continfo
```

IMÁGENES DOCKER

5.1 docker_images/bajo_consumo/Dockerfile

```
dockerfile
FROM alpine:latest
CMD ["sleep", "infinity"]
```

5.2 docker_images/alto_ram/Dockerfile

```
dockerfile
FROM alpine:latest
RUN apk add --no-cache stress-ng
CMD ["stress-ng", "--vm", "1", "--vm-bytes", "512M", "--vm-keep"]
```

5.3 docker_images/alto_cpu/Dockerfile

```
dockerfile
FROM alpine:latest
RUN apk add --no-cache stress-ng
CMD ["stress-ng", "--cpu", "4"]
```

5.4 docker_images/intruso/Dockerfile

dockerfile

FROM alpine:latest

CMD ["sh", "-c", "while true; do cat /etc/passwd 2>/dev/null; sleep 5; done"]

5.5 Construir las 4 imágenes

bash

cd ~/workspaces/SistemasOperativos1/Proyecto2/docker_images

docker build --label proyecto=so1 --label tipo=bajo \
-t so1_bajo:latest ./bajo_consumo

docker build --label proyecto=so1 --label tipo=alto_ram \
-t so1_alto_ram:latest ./alto_ram

docker build --label proyecto=so1 --label tipo=alto_cpu \
-t so1_alto_cpu:latest ./alto_cpu

docker build --label proyecto=so1 --label tipo=intruso \
-t so1_intruso:latest ./intruso

5.6 Verificar

bash

docker images | grep so1

Debe aparecer las 4 imágenes.

VALKER

6.1 Levantar el contenedor

bash

docker run -d \
--name valkey_metrics \
-p 6379:6379 \
valkey/valkey:7.2-alpine

6.2 Verificar

bash

redis-cli -h localhost -p 6379 PING

Debe responder PONG.

bash

redis-cli -h localhost -p 6379 KEYS '*'

GRAFANA

7.1 Levantar Grafana

bash

```
docker run -d \
  --name so1-grafana \
  --restart unless-stopped \
  -p 3000:3000 \
  -e GF_SECURITY_ADMIN_USER=admin \
  -e GF_SECURITY_ADMIN_PASSWORD=admin \
  -e GF_INSTALL_PLUGINS=redis-datasource \
  grafana/grafana:latest
```

Conectar Grafana a la red de Valkey

Descubrir la red de Valkey:

bash

```
docker inspect valkey_metrics --format '{{range $k, $v :=
.NetworkSettings.Networks}}{{ $k }}{{ "\n" }}{{ end }}
```

Debe salir algo como proyecto2_monitoring_net.

Conectar Grafana:

bash

```
docker network connect NOMBRE_DE_LA_RED so1-grafana
```

Verificar resolución DNS:

bash

```
docker exec so1-grafana getent hosts valkey_metrics
```

Debe salir 172.x.x.x valkey_metrics.

Configurar datasource en Grafana

1. Abrir <http://localhost:3000>
2. Login: admin / admin
3. Menú lateral → Connections → Data sources → Add data source → Redis
4. Configurar:
 - Name: Valkey
 - Address: valkey_metrics:6379
 - Password: (vacío)
5. Save & Test → debe decir "Data source is working"

SCRIPT DEL CRONJOB

8.1 scripts/create_containers.sh

bash

nano ~/workspaces/SistemasOperativos1/Proyecto2/scripts/create_containers.sh

Pega:

bash

#!/bin/bash

LOG="/tmp/cron_so1.log"

FECHA=\$(date '+%Y-%m-%d %H:%M:%S')

log() { echo "[\$FECHA] \$1" >> "\$LOG"; }

count_label() {

docker ps --filter "label=proyecto=so1" --filter "label=tipo=\$1" -q 2>/dev/null | wc -l

}

BAJOS=\$(count_label bajo)

ALTOS_RAM=\$(count_label alto_ram)

ALTOS_CPU=\$(count_label alto_cpu)

ALTOS=\$((ALTOS_RAM + ALTOS_CPU))

INTRUSOS=\$(count_label intruso)

log "Estado: bajos=\$BAJOS altos=\$ALTOS intrusos=\$INTRUSOS"

FALTAN=\$((3 - BAJOS))

for i in \$(seq 1 \$FALTAN); do

[\$FALTAN -le 0] && break

ID=\$(docker run -d --rm --label proyecto=so1 --label tipo=bajo so1_bajo:latest)

log "Creado bajo: \${ID:0:12}"

done

FALTAN=\$((2 - ALTOS))

for i in \$(seq 1 \$FALTAN); do

[\$FALTAN -le 0] && break

if [\$((RANDOM % 2)) -eq 0]; then

ID=\$(docker run -d --rm --label proyecto=so1 --label tipo=alto_ram so1_alto_ram:latest)

log "Creado alto RAM: \${ID:0:12}"

else

ID=\$(docker run -d --rm --label proyecto=so1 --label tipo=alto_cpu so1_alto_cpu:latest)

log "Creado alto CPU: \${ID:0:12}"

fi

done

if ["\$INTRUSOS" -eq 0] && [\$((RANDOM % 5)) -eq 0]; then

ID=\$(docker run -d --rm --label proyecto=so1 --label tipo=intruso so1_intruso:latest)

log "Creado intruso: \${ID:0:12}"

fi

```
FINAL_BAJOS=$(count_label bajo)
FINAL_ALTOS=$(( $(count_label alto_ram) + $(count_label alto_cpu) ))
log "Final: bajos=$FINAL_BAJOS altos=$FINAL_ALTOS"
bash
chmod +x ~/workspaces/SistemasOperativos1/Proyecto2/scripts/create_containers.sh
```

8.2 Probar manualmente

```
bash
~/workspaces/SistemasOperativos1/Proyecto2/scripts/create_containers.sh
docker ps --format "table {{.Image}}\t{{.Status}}"
Debe crear 3 bajos + 2 altos.
```

DAEMON EN GO

9.1 Inicializar el módulo Go

```
bash
cd ~/workspaces/SistemasOperativos1/Proyecto2/daemon
go mod init so1/daemon
go get github.com/go-redis/redis/v8
```

9.2 Crear daemon/cmd/daemon/main.go

```
bash
nano cmd/daemon/main.go
Pega el código completo del daemon (versión con lógica de matar contenedores).
```

9.3 Compilar

```
bash
cd ~/workspaces/SistemasOperativos1/Proyecto2/daemon
go build -o bin/so1-daemon ./cmd/daemon
ls -la bin/so1-daemon
```

9.4 Ejecutar

```
bash
nohup ./bin/so1-daemon > /tmp/daemon.log 2>&1 &
sleep 5
tail -10 /tmp/daemon.log
Debe aparecer:
text
Valkey conectado
Cronjob instalado
RAM: total=2849MB libre=xxxMB uso=xxxMB procesos=xxx
Contenedores: bajos=3 altos=2
```

9.5 Verificar cronjob instalado

```
bash
```

```
crontab -l
```

Debe aparecer la línea con create_containers.sh.

VERIFICACIÓN FINAL

10.1 Checklist completo

```
bash
```

```
echo "=== 1. MODULO KERNEL ==="
```

```
lsmod | grep continfo
```

```
ls -la /proc/continfo_pr2_so1_201700698
```

```
echo ""
```

```
echo "=== 2. DAEMON ==="
```

```
pgrep -f so1-daemon && echo "corriendo"
```

```
echo ""
```

```
echo "=== 3. CRON ==="
```

```
crontab -l
```

```
echo ""
```

```
echo "=== 4. CONTENEDORES ==="
```

```
docker ps --format "table {{.Image}}\t{{.Status}}"
```

```
echo ""
```

```
echo "=== 5. VALKER ==="
```

```
redis-cli -h localhost -p 6379 PING
```

```
redis-cli -h localhost -p 6379 KEYS 'so1:*
```

```
echo ""
```

```
echo "=== 6. GRAFANA ==="
```

```
docker ps | grep grafana
```

10.2 Salida esperada

- Módulo: aparece en lsmod y el /proc existe
- Daemon: PID activo
- Cron: línea con create_containers.sh
- Contenedores: 3 bajos + 2 altos (más intruso ocasional)
- Valkey: PONG y varias claves so1:*
- Grafana: contenedor corriendo

10.3 Acceder al dashboard

1. Navegador: <http://localhost:3000>

2. Login: admin / admin
3. Dashboard: PROYECTO2 SO1

OPERACIÓN DIARIA

11.1 Arrancar todo (después de reiniciar)

bash

1. Módulo kernel

cd ~/workspaces/SistemasOperativos1/Proyecto2/kernel_module

sudo insmod continfo.ko

2. Verificar Valkey y Grafana (si están detenidos)

docker start valkey_metrics so1-grafana 2>/dev/null

3. Daemon

cd ../daemon

nohup ./bin/so1-daemon > /tmp/daemon.log 2>&1 &

11.2 Detener todo

bash

Daemon (elimina cronjob automáticamente)

kill -f so1-daemon

Módulo kernel

sudo rmmod continfo

Contenedores de prueba

docker rm -f \$(docker ps -q --filter "label=proyecto=so1")

11.3 Monitoreo

bash

Log del daemon

tail -f /tmp/daemon.log

Log del cron

tail -f /tmp/cron_so1.log

Log del kernel

sudo dmesg -w

Contenedores

watch -n 2 'docker ps --filter label=proyecto=so1'

SOLUCIÓN DE PROBLEMAS

12.1 El módulo no compila

bash

Verificar headers

ls /lib/modules/\$(uname -r)/build

Si no existen (WSL2 con kernel custom):

ls /home/tj/wsl-kernel/

Debe existir Makefile, include/, etc.

12.2 El módulo no carga

bash

sudo insmod continfo.ko

sudo dmesg | tail

Errores comunes:

- File exists: sudo rmmod continfo primero
- Unknown symbol: headers incorrectos

12.3 Docker sin permisos

bash

sudo usermod -aG docker \$USER

newgrp docker

12.4 Grafana no conecta a Valkey

bash

Verificar misma red

docker inspect valkey_metrics --format '{{range \$k, \$v :=
.NetworkSettings.Networks}}{{ \$k }}{{ "\n" }}{{ end }}

docker inspect so1-grafana --format '{{range \$k, \$v :=
.NetworkSettings.Networks}}{{ \$k }}{{ "\n" }}{{ end }}

Si no coinciden, conectar

docker network connect NOMBRE_RED so1-grafana

12.5 El cronjob no se ejecuta

bash

Verificar cron corriendo

pgrep -x cron

sudo service cron start

12.6 Contenedores huérfanos

bash

Matar todos los so1 sin label

```
for img in so1_bajo so1_alto_ram so1_alto_cpu so1_intruso; do
    docker ps -q --filter "ancestor=${img}:latest" | while read id; do
        label=$(docker inspect --format '{{index .Config.Labels "proyecto"}}' $id)
        [ "$label" != "so1" ] && docker rm -f $id
    done
done
```